

GPU implementation of the 2D shallow water equations for the simulation of rainfall/runoff events

Asier Lacasta · Mario Morales-
Hernández · Javier Murillo · Pilar
García-Navarro

Received: date / Accepted: date

Abstract Hydrological processes that occur in catchments usually require large space resolution over long periods of time. The advance on numerical methods as well as the increasing power of computation are making possible the physically based simulation of these phenomena. In particular, the 2D shallow water equations can be used to provide distributions of water depth and velocity fields. The necessity of spatial resolution involves the use of a large number of elements hence increasing the computational time when simulating realistic scenarios for a long time period. This work deals with an efficient GPU implementation of the 2D shallow water equations on unstructured meshes analysing the influence of the mesh resolution both on the computational performance and the quality of the results to simulate a rainfall/runoff event. The numerical method to solve them has been developed and compared following three programming approaches: the sequential implementation and its adaptation to the the Multi-Thread and Many-Core architectures. The particular detail of the influence of the mesh ordering when using unstructured triangular meshes is paid attention in this work in order to find the best strategy to further reduce the computational time in the context of GPU simulation. The resulting approach is efficient and can become very useful in environmental simulation of Hydrological processes.

Keywords shallow water equations · GPU · Unstructured Meshes · rainfall/runoff · wet/dry cells

A. Lacasta
LIFTEC-EINA, CSIC-Universidad Zaragoza C/ Maria de luna 3
Zaragoza, SPAIN Tel.: +34-976-555052
E-mail: alacasta@unizar.es

1 Introduction

Numerical methods used as computational engine of hydrological models usually require a large computational effort in the background of their calculations. Domains of interest in Hydrology are usually large and, when the interest is in rainfall-runoff events, the problem is controlled by the wet and dry fronts over irregular topography. The more precision and information is required, the more complicated these models must be. At the same time, efficient calculations for practical applications are required to be completed in an affordable time. On the other hand, the application of physically based mathematical models such as the two dimensional shallow water equations (SWE) for the simulation of rainfall/runoff phenomena provides a very useful manner to describe the flow in detail. The main drawback of the application of these equations is the large computational effort that they require. Moreover, when more phenomena such as infiltration are included, the calibration of parameters are also required, needing several runs to adjust them.

Nowadays Moore's law which predicts that transistor capacity is doubled every two years is being replaced by the modified version of the statement "the software has to also start following Moore's law, software has to double the amount of parallelism that it can support every two years" since the number of cores in a processor is set to double every 18 months (Holwerda, 2007).

Parallelism can be exploited by different approaches. The most common and straightforward parallelization technique is on Shared Memory machines. This kind of parallelization can be handled by the OpenMP standard that is actually implemented on most of the programming languages. It is important to note that parallel operations based on shared memory space may require special implementations of the algorithm in some cases to avoid data-dependency hazards. Taking into account that nowadays almost every processor is built under the Multi-Core pattern, OpenMP allows to enhance in a simple manner the parallel model to most of the common processors. On the other hand, it is possible to export the parallel model to Distributed-Memory structures or heterogeneous architectures using standards such as the MPI protocol. The design and implementation of the parallel solutions are not as simple as in the case where memory space is shared between the processing units. Distributed-Memory implementations require to be careful with communication lapses as well as to optimize most of the data transactions between every pair of elements in the network. Moreover, these implementations usually require large investment, being this justified only for very high intensive computing requirements. Nevertheless, some advances on the application of these techniques for the 2D shallow water equations have been presented in recent years (Sanders et al., 2010). The main difficulty for these implementations is to adjust the workload as well as to decompose the domain in the most suitable manner in order to minimize the communication between the subdomains. In Lacasta et al. (2013) a one dimensional strategy for the domain decomposition is presented, showing reasonable performance. In Hinkelmann (2005) a more sophisticated strategy for the domain decomposition is described. That strategy uses the

idea of reordering the connectivity matrix using the methods that this work deals with.

As an alternative, Stream processors (also known as Many-Core processors) have appeared recently. One of the most representative architecture that implements this idea is the modern Graphical Processing Units (GPUs). This hardware facility allows developers the use of a large number of simplified computing elements that can process a large number of mathematical operations all together and at the same time. The fundamental idea is to reduce the possibility of each processing unit (or core) by simplifying the electronics and the energy consumption. This allows to extend the Multi-Core implementation to a simplified version in terms of computing capability but providing the possibility of integrating a lot of these elements on the same chip. The main disadvantage of the architecture is the necessity of providing good memory patterns because of its inherent coalesced building. In other words, once the calculations are optimized at the maximum, the next step when tuning the implementations is related to the way the information is located. When dealing with structured meshes or grids, the ordering is helpful to maximize the performance of the GPU implementation. The GPU alternative has proved efficient in the context of Fluid Dynamics in general for the resolution of the 3D Navier-Stokes (NS) equations (Chandar et al., 2013; Thibault and Senocak, 2009). The shallow water equations, being a depth averaged formulation of the NS equations are usually preferred for environmental simulations over large areas. Advances on the implementations of the shallow water equations with these meshes have been described in Brodtkorb et al. (2012). Additionally, unstructured grid GPU based implementations of these equations was described in Castro et al. (2011), showing important performance on their implementations. Nevertheless, in order to increase this performance, some useful techniques such as the mesh reordering can be applied (Giles and Burgess, 1997). In the case of the edge-based solvers, there is an additional issue to be taken into account when optimized memory accessing is required and it is that not only the cells but also the edges play an important role in the performance (Aubry et al., 2011). These optimizations are even more important when calculations are made on modern GPUs. These recent architectures usually require coalescing in the data storage to maximize the performance of the memory transactions as well as to reduce at the most the time required in the execution of the kernels. Recently some technique for matrix bandwidth reduction (Cuthill–McKee) has been applied in this kind of solvers (Lacasta et al., 2014a,b) showing an important increase in the performance of the implementation. When designing for GPU the most used frameworks are OpenCL and CUDA. OpenCL provides a high-level set of instructions that allow to the programmer to design parallel implementations across heterogeneous systems. This makes possible to run the same code on both GPU as well as on CPU exploiting it parallelism as in OpenMP. A very interesting implementation of the shallow water equations on Cartesian grids using OpenCL is detailed in Smith and Liang (2013). On the other hand, CUDA is a programming model

designed by NVIDIA to run parallel solutions on their GPUs. In this work, the GPU implementation of the numerical method has been made using CUDA.

In this work, both Multi-Core and Many-Core implementations, already developed and detailed in a previous paper, are tested in the simulation of rainfall/runoff events comparing their performance against a simplified not parallelized version, showing the possibilities that the parallel techniques can offer to the users and developers of similar models. Additionally, the improvement introduced by a new connectivity matrix reordering (Sloan's method) will be evaluated. The structure of the text is as follows: First, the formulation of the shallow water flow model used is summarized and the basic features of the finite volume method applied to solve the equations is outlined. Then an overview of the GPU implementation is provided with special emphasis on the difficulties associated to unsteady flow on unstructured meshes. The rainfall/runoff test case is then presented and the results obtained using different computational implementations are compared and discussed.

2 Mathematical model & Numerical method

The mathematical model considered for the simulation is based on the shallow water equations, where the general three-dimensional conservation laws are depth averaged. Their formulation, when neglecting viscous/turbulent terms as well as Coriolis forces can be written in conservative form as follows

$$\frac{\partial \mathbf{U}}{\partial t} + \frac{\partial \mathbf{F}(\mathbf{U})}{\partial x} + \frac{\partial \mathbf{G}(\mathbf{U})}{\partial y} = \mathbf{S}(\mathbf{U}) \quad (1)$$

where

$$\mathbf{U} = (h, hu, hv)^T \quad (2)$$

are the conserved variables with h representing the water depth (m), (hu, hv) the depth averaged components of the unit discharge vector (m^2/s) and (u, v) the depth averaged components of the velocity vector (m/s). The fluxes of these variables, $(\mathbf{F}, \mathbf{G})$ are given by:

$$\mathbf{F} = \left(q_x, \frac{q_x^2}{h} + \frac{1}{2}gh^2, \frac{q_x q_y}{h} \right)^T, \quad \mathbf{G} = \left(q_y, \frac{q_x q_y}{h}, \frac{q_y^2}{h} + \frac{1}{2}gh^2 \right)^T \quad (3)$$

where g is the acceleration due to the gravity (m/s^2). The source terms of the system are the precipitation rate, the bed slope and the friction terms:

$$\mathbf{S} = (i_e, gh(S_{0x} - S_{fx}), gh(S_{0y} - S_{fy}))^T \quad (4)$$

where i_e is the effective precipitation rate (m/s), $S_{0x} = \frac{\partial z}{\partial x}$ and $S_{0y} = \frac{\partial z}{\partial y}$ are the (dimensionless) components of the bed slope, being z the bed level

(m) and S_{fx} and S_{fy} are the (dimensionless) components of the friction slope, written in terms of the Gauckler-Manning's roughness coefficient n ($sm^{-1/3}$):

$$S_{fx} = \frac{n^2 u \sqrt{u^2 + v^2}}{h^{4/3}}, \quad S_{fy} = \frac{n^2 v \sqrt{u^2 + v^2}}{h^{4/3}} \quad (5)$$

In order to obtain a numerical solution from a finite volume approach, equation (1) is integrated over each computational cell Ω_i with area A_i :

$$\frac{\partial}{\partial t} \int_{\Omega_i} \mathbf{U} d\Omega + \int_{\Omega_i} (\vec{\nabla} \mathbf{E}) d\Omega = \int_{\Omega_i} \mathbf{S} d\Omega \quad (6)$$

with $\mathbf{E} = (\mathbf{F}, \mathbf{G})$ and using the Gauss theorem

$$\frac{\partial}{\partial t} \int_{\Omega_i} \mathbf{U} d\Omega + \oint_{\Gamma} \mathbf{E} \mathbf{n} d\Gamma = \int_{\Omega_i} \mathbf{S} d\Omega \quad (7)$$

where $\mathbf{n} = (n_x, n_y)$ is the outward unit normal vector to the volume Ω_i and Γ denotes the surface surrounding the volume Ω_i . It is possible to define a Jacobian matrix $\mathbf{J}_{\mathbf{n}}$ of the normal flux $\mathbf{E} \mathbf{n}$ and to diagonalise it in terms of matrices $\mathbf{\Lambda}$ and $\mathbf{P}$

$$\mathbf{J}_{\mathbf{n}} = \frac{\partial(\mathbf{E} \mathbf{n})}{\partial \mathbf{U}} = \mathbf{P} \mathbf{\Lambda} \mathbf{P}^{-1} \quad (8)$$

where the eigenvalues and eigenvectors play an important role. Roe's linearization Roe (1985) allows us to define locally an approximate matrix $\tilde{\mathbf{J}}_{\mathbf{n}}$ at each wall k whose eigenvalues $\tilde{\lambda}^m$ and eigenvectors $\tilde{\mathbf{e}}^m$ can be used to express the differences in vector $\mathbf{U}$ and the discrete representation of the source term across the grid edge k as a sum of waves Murillo et al. (2009b):

$$\delta \mathbf{U}_k = \mathbf{U}_i - \mathbf{U}_j = \sum_m (\tilde{\alpha} \tilde{\mathbf{e}})_k^m \quad (9)$$

where i and j are the indexes of the cells sharing that share the edge k . This enables:

$$\delta \mathbf{E} \mathbf{n}_k = \mathbf{J}_{\mathbf{n}} \delta \mathbf{U}_k = \mathbf{J}_{\mathbf{n}} \sum_m (\tilde{\alpha} \tilde{\mathbf{e}})_k^m \quad (10)$$

The source terms (neglecting the rain rate) are also projected onto the eigenvectors basis:

$$\mathbf{S}_k = \sum_m (\tilde{\beta} \tilde{\mathbf{e}})_k^m \quad (11)$$

The particular form of the eigenvectors, eigenvalues, α and β coefficients can be found in Murillo et al. (2009b) and is not repeated here for the sake of brevity. Finally, using a first order upwind scheme and the flux-splitting formulation based on the above components, it is feasible to write the updating of a single cell i according to the arriving contributions from the neighbouring edges:

$$\mathbf{U}_i^{n+1} = \mathbf{U}_i^n - \frac{\Delta t}{A_i} \sum_{k=1}^{NE} \sum_{m=1}^3 (\tilde{\lambda}^- \tilde{\gamma} \tilde{\mathbf{e}})_k^m l_k \quad (12)$$

where $\tilde{\lambda}_k^{\pm, m} = \frac{1}{2}(\tilde{\lambda} \pm |\tilde{\lambda}|)_k^m$, $\tilde{\gamma}_k = \left(\tilde{\alpha} - \frac{\tilde{\beta}}{\tilde{\lambda}}\right)_k$ express compactly the fluxes and source terms contributions of each edges k of length l_k and NE denotes the number of neighbouring cells ($NE = 3$ for triangular cells) Morales-Hernández et al. (2013).

Being an explicit scheme, the time step is restricted by stability reasons. When considering 2D unstructured meshes, it is possible to define the following quantities χ_i for each cell i , as an estimation of a characteristic distance involving the volume of the cell and the length of the shared k edges :

$$\chi_i = \frac{A_i}{\max_{k=1, NE} l_k} \quad (13)$$

Therefore, the time step size must be chosen according to:

$$\Delta t = CFL \min_k \left\{ \frac{\min(\chi_i, \chi_j)}{\max_m |\tilde{\lambda}_k^m|} \right\} \quad CFL \leq 0.5 \quad (14)$$

The previous stability condition is insufficient in presence of relatively important source terms. The systematic control of numerical stability in those cases has been a matter of recent research Murillo and García-Navarro (2010) as it is related with the applicability of the scheme to real situations. A simple generalization of the CFL condition paying attention to the existence of the source terms can lead to extremely small values of Δt various orders of magnitude smaller than the value dictated by the homogeneous condition, hence rendering the method impractical. This is crucial when applying these methods for hydrological purposes where dynamically wet-dry fronts are established. Figure 1 compares both hydraulic and hydrologic wetting and drying situations. The main difference between both is the null existence of a clear wet/dry line front when simulating rainfall events in hydrological cases. Whereas in typical hydraulic problems the wet/dry front is a boundary, in hydrological problems, wet/dry frontiers may be found everywhere.

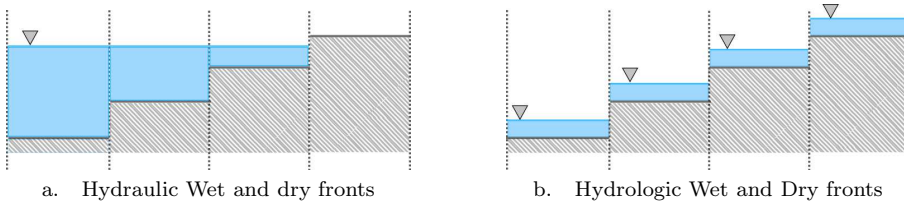


Fig. 1 Comparison of typical situations of wetting and drying from the hydraulic (a) and hydrological (b) perspective

This constant wetting and drying situations over all the cells implies that special care must be taken when calculating wet/dry fronts Murillo and García-Navarro (2010). Moreover, it has been shown that impractical reduction of the time-step can be avoided in these cases by means of a reconstruction of the approximate solution Murillo and García-Navarro (2010) that is not detailed here for the sake of conciseness. The strategy proposed is based on enforcing positive values of interface discrete water depths coming from a detailed study of the Riemann Problem. When they become negative, the numerical contribution of the friction and bed slope source term is reduced instead of reducing the time step size. For more details, see Murillo et al. (2009a); Murillo and García-Navarro (2010).

Furthermore, it is important to emphasize the importance of using a unified discretization of fluxes and source terms together with a careful discrete formulation of the bed slope terms. In the context of a piecewise formulation of all the variables, the bed level representation in an irregular basin implies the existence of bed discontinuities all over the domain. Also, in hydrological conditions, these bed discontinuities are combined with small values of water depth that, in turn, are also discontinuous. As already detailed Murillo and García-Navarro (2010), this type of scenario requires the application of a careful discretization of the bed slope source term that introduces an integral formulation of the pressure force on the bed discontinuity.

The discretization of the friction term based on Murillo et al. (2009a) is applied

$$S_{f,k} = \left(\frac{n^2 \tilde{\mathbf{u}} \mathbf{n} |\tilde{\mathbf{u}}|}{\max(h_i, h_j)^{4/3}} \right)_k \quad (15)$$

On the other hand, the precipitation rate i_e is updated following a centered philosophy, i.e, once the time step size Δt is computed, the quantity $i_e \Delta t$ is added to each cell in the form of mass of water:

$$h_i = h_i + i_e \Delta t \quad (16)$$

3 GPU implementation

The necessity of both large number of time-steps because of the CFL condition as well as the large number of cells required to capture all the topographic details make an efficient implementation of the method indispensable. Nowadays most of the processors are designed with multi-threading capabilities. Nevertheless, it is not direct to perform operations using every single core. Figure 2 displays a typical iterative process using 1 core of a Multi-Core processor.

When possible, the iterative process may be split into different sub-processes allowing the processing of different elements at the same time by different units. This is feasible when the loop contains straight-line code (a single basic block with no jumps) and when there is no data dependency between iterations (element $i - 1$ is not required for the element i). If data flow does not

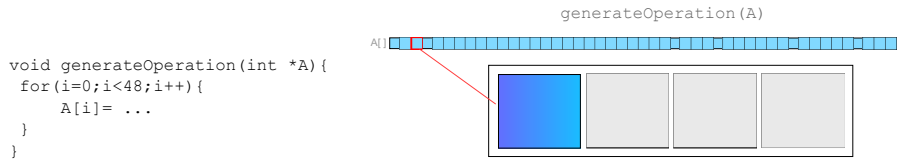


Fig. 2 Sequential iteration over vector `A[]`

satisfy the previous condition, the manner of processing the elements to make them parallelizable must be re-structured.

OpenMP is a very useful standard to perform shared-memory parallelization. This standard implements parallel primitives in many programming languages such as C or Fortran making it really easy to implement a parallel solution in a reasonable time. The main disadvantage of this kind of parallelization is that the Multi-Core processors are not growing as much as the most recent Many-Core architectures, where a lot of specialized processing units allow to make many more operations than in the multi-core unit. An example of a parallel loop using OpenMP is displayed in Figure 3.

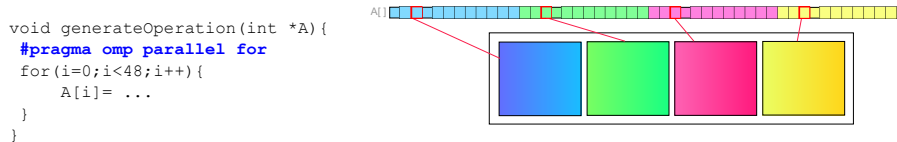


Fig. 3 Parallel iteration over vector `A[]` using 4 cores

Many-Core processors provide a hardware architecture to perform a large number of independent operations in a parallel manner. The main differences between Multi-Core threads and Many-Core threads are the lightweight of the second solution as well as the hardware support of a larger number of the latter (> 1000s). Unlike the Multi-Core thread, where each thread processes a group of elements, in the Many-Core paradigm, each thread will process just one element.

CUDA (Compute Unified Device Architecture) was released in order to perform this kind of parallelization on the NVIDIA's GPUs. It provides both a software model and a set of compilation tools that support the NVIDIA Many-Core GPUs. An example of a CUDA instance for the previous loop is displayed in Figure 4.

4 Dealing with unstructured meshes

Unstructured meshes have demonstrated good properties for the simulation of rainfall/runoff events (Caviedes-Voullieme et al., 2012). The manner that the cells are connected does not establish preferential directions and this reduces

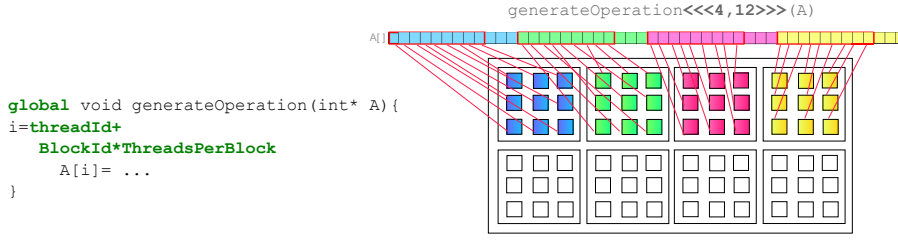


Fig. 4 Massively parallel iteration over vector $A[]$ using a Many-Core architecture.

the mesh influence on the numerical results. Also, they are well adapted to local refinement.

The disadvantage of using unstructured meshes on GPU is related with the mesh disorder. It becomes impossible to establish an ordered pattern of accessing the cells and it is necessary to improve the way the threads operate with the elements because of the coalescence necessity. There are two important aspects that must be taken into account. The computation of the fluxes is made by edges while the updating process is performed looping by cells. In (Lacasta et al., 2014a) some details about this issue are explained and it is possible to improve these operations reordering the cell connectivity. This reordering results in the reduction of the bandwidth property. This property can be defined as follows

$$\varphi = \max\{|f(v_i) - f(v_j)| : v_i v_j \in E\} \quad (17)$$

where $|f(v_i) - f(v_j)|$ defines the distance between elements v_i and v_j . For our purpose, $f(v_i)$ is the index of the element v_i and the difference of the indexes of two neighbouring elements v_i and v_j represents the distance in memory allocation for those elements.

The reduction of the matrix bandwidth is usually obtained by means of vertex ordering algorithms. In (Mueller et al., 2007) some of them are benchmarked as well as analysed for different purposes. In our case, the most important aspect is the reduction of the bandwidth but there are other interesting properties that can be improved such as the profile which is interesting in other tasks as the computation for solving linear equations or eigenvalue problems (Gibbs et al., 1976).

The application of these algorithms was explored in Hinkelmann (2005) to design a suitable manner of decomposing computational domains for distributed parallel numerical implementations. In addition, some of these methods have been previously tested for these purposes in other works such as (Aubry et al., 2011; Giles and Burgess, 1997). They show the positive effect that the bandwidth reduction has over the memory access, improving the number of hits when reading cache entries on conventional architectures. In particular, in Lacasta et al. (2014a) the RCM (Reverse Cuthill–McKee) algorithm is analysed as a solution to improve the performance of the 2D shallow water equations implementation on GPU. The RCM algorithm is a variant of

the Cuthill–McKee algorithm (Cuthill and McKee, 1969) where the resulting index numbers is reversed.

Inspired by (Aubry et al., 2011), the Sloan ordering algorithm (Sloan, 1986) is included in this new test. This algorithm not only improves the bandwidth but also reduces both the profile and the wave front. These properties seem not to be important for explicit solvers but a different clusterization pattern is produced. This clusterization, in contrast to Cuthill–McKee algorithm, takes local as well as global criteria for the renumbering sequence into account.

5 Test case

The test case selected for the evaluation of the implementation was described in (Caviedes-Voullieme et al., 2012) where more details about the influence of the meshes in rainfall/runoff simulations can be found. Because of the sensitivity of the meshes, the suitability of using unstructured meshes is proven and then the optimized implementation on GPUs is justified.

The simulated event took place in the Arnás catchment, located in the northern Spain Pyrenees (Figure 5, with a total surface of 2.84 km^2 and an altitude which varies from 900 to 1340 m.a.s.l. A measured rain series of 8 hours in October 2005 is considered and simulated taking into account a uniform friction coefficient obtained by averaging the roughness map reported in (Serrano-Pacheco, 2009). This parameter may be calibrated using the outflow response time (Mateo et al., 2014).

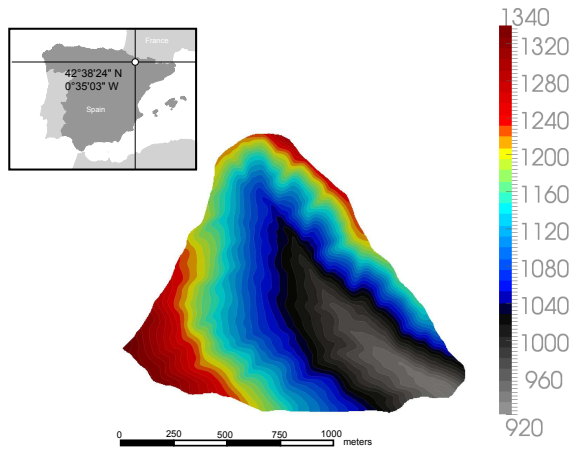


Fig. 5 Height map (m) of the Arnás catchment.

The event has been simulated using two meshes (Figure 6). The first one labelled as *refined* is a 31000 cells locally refined mesh. The refinement criterion

is the slope gradient following the idea of (Caviedes-Voullieme et al., 2012). The second one named *uniform* is a > 800000 cells isotropical mesh. Both meshes are built following the Delaunay triangularization.

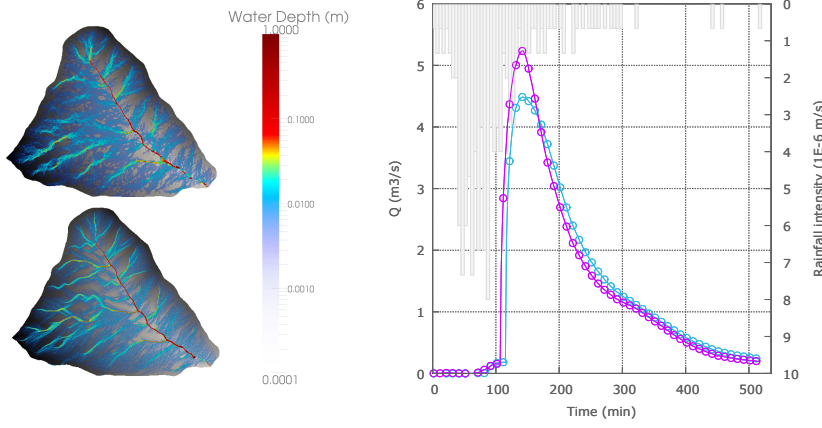


Fig. 6 Comparison of numerical results of water depth (left) at $t = 5800s$ for both *refined* (upper) and *uniform* (lower) benchmarked meshes and the comparison of the outlet discharge for both meshes at the right side for the locally refined mesh (-o-) and for the uniform mesh (-o-) (right)

The meshes have been chosen to analyse two important aspects. The first one is the quality of the results of the meshes for hydrological purposes and the second one is the influence of the size of the problem in the performance of the GPU implementation.

5.1 Quality of the numerical results

In general, hydrological processes deal with complex phenomena such as erosion and infiltration. The former is not taken into account in the present model. On the other hand, infiltration phenomena can be modelled using empirical formulations. In the present approach, infiltration has been formulated with a simple method such as the CN SCS method following the previous work in Caviedes-Voullieme et al. (2012) which modifies the precipitation $P_t(mm)$ into an effective precipitation $P_e(mm)$ as

$$P_e = \frac{(P_t - \alpha S)^2}{P_t + S(1 - \alpha)}, \quad S = \frac{25400}{CN} - 254 \quad (18)$$

in terms of the maximum soil retention capacity $S(mm)$, related to the dimensionless CN parameter, and the coefficient α used to define the initial abstraction $I_a = \alpha S(mm)$. The values used for this case are the curve number $CN = 90.1$ and the initial abstraction ratio $\alpha = 10^{-4}$. The effective precipitation can be included in (16) as a velocity dividing by Δt as $i_e = P_e/\Delta t$

taking into account differences in that effective precipitation between t^{n-1} and t^n . It is very important to take into account that these parameters are usually calibrated as suggested in (Caviedes-Voullieme et al., 2012), where several simulations are required to adjust them. Other formulations such as Horton or Green-Ampt methods (Esteves et al., 2000) for example could also be used. Figure 6 shows how both meshes obtain a very similar result in terms of water depth distribution and outlet discharge. Both variables are the most representative variables in the infiltration modelling so both may provide very similar results when including this effect.

On the other hand, velocity is not captured as in the case of the water distribution. The way the mesh is discretized has a very important effect on how the velocity field is distributed. Figure 7 shows the velocity magnitude map and the local value of this variable at $p_1 = [696616.1, 4724223.4]$ and $p_2 = [696704.7, 4724860.1]$ as well as their locations. This variable shows more sensitivity to the mesh and to the size of the cells and their distribution.

This could be of interest in further developments including bed erosion and sediment deposition (Juez et al., 2014).

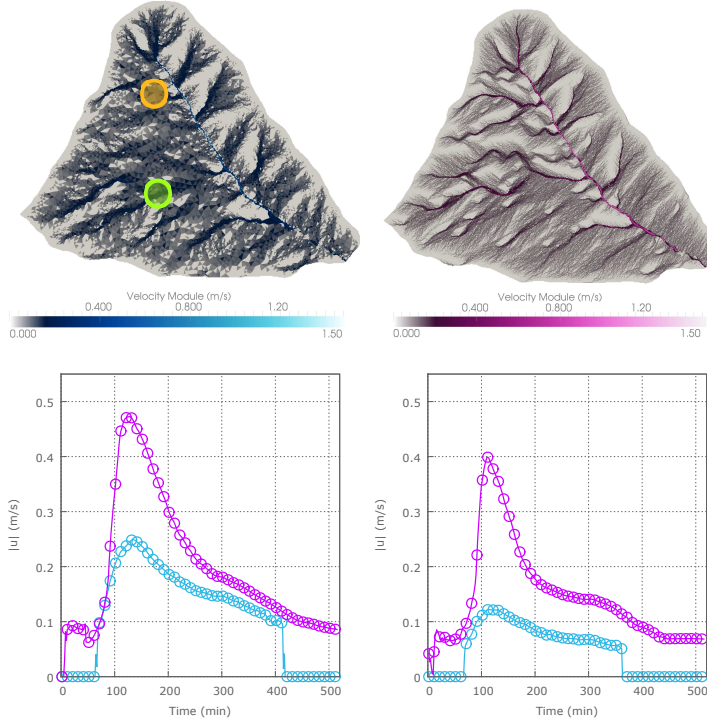


Fig. 7 Comparison of numerical results of velocity magnitude at $t = 5800s$ for both *refined* (upper-left) and *uniform* (upper-right) benchmarked meshes. Probes locations are detailed in the refined mesh with (●) for the probe located at p_1 and with (●) for probe located at p_2 . Comparison of velocity magnitude for both meshes (locally refined (---○) and uniform (---○)) for two different locations at the mesh p_1 (bottom-left) p_2 (bottom-right)

5.2 Performance of the computational implementation

From the computational performance point of view, three implementations have been tested. The first one corresponds to a sequential CPU standard implementation and this will be the reference computational cost with which the other implementations will be compared against. The second one is an OpenMP implementation of the same code. Both implementations have been run on a last generation *Intel Core i7 4770 @3.40 GHz CPU*. The third implementation corresponds to a GPU implementation very similar to the one presented in (Lacasta et al., 2014a) and (Lacasta et al., 2014b) and it has been implemented using the CUDA tool kit. This GPU implementation has been tested on two different nVidia GPUs. One corresponds to the previous generation Tesla family GPU *nVidia Tesla c2075*. This GPU was released on Oct. 2011 and it is based on the *Fermi* Micro-architecture. The other GPU is a *nVidia Titan Black* and it is based on the next generation *Kepler* architecture. The memory bandwidth of the latter is three times greater than for the Tesla c2075. Moreover, the overall peak performance of the *nVidia Titan Black* is more than two times better. Despite each implementation requires little modifications on the implementation due to language requirements, the three implementations have been designed to run on the same conditions. In other words, from the algorithmic point of view, the three implementations are similar with slightly differences due to the adaptation to each framework.

Case	n_{cells}	Standard Mesh	Sloan Mesh		RCM Mesh	
		Bandwidth	Bandwidth	time (s)	Bandwidth	time (s)
<i>Refined</i>	31662	30989	2389	3.931	275	0.740
<i>Uniform</i>	868241	868241	4862	455.38	971	25.001

Table 1 Detail of the mesh properties for the three different mesh orderings and the computational cost of the renumbering operation for the Sloan and RCM strategies.

As suggested in (Lacasta et al., 2014b), the application of reordering strategies on the mesh can increase the performance of the computation. In order to analyse this effect, two different algorithms have been applied and three different orderings are tested: the standard ordering generated by the mesher, Sloan’s ordering and the RCM ordering. These techniques have produced different patterns for the two meshes. The computational effort for both ordering methods has been also compared. Table 1 shows that Sloan’s ordering requires a larger computational effort to perform the numbering operation. On the other hand Table 1 also shows that in the case of the refined mesh, the original bandwidth is reduced in a 92% using Sloan’s reordering method while in the case of the RCM the bandwidth was reduce to near the 97%. Figure 8 shows the detail of the composition of the connectivity matrix as well as the distribution of the distances defined in (17). In the case of the uniform mesh, Table 1 shows that the reduction of the mesh bandwidth using Sloan’s ordering reaches a 99.4% factor while in the case of the RCM the bandwidth

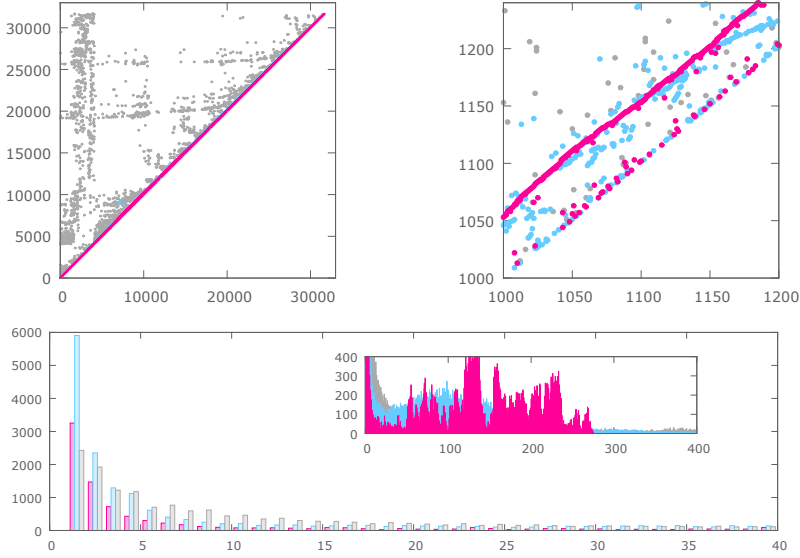


Fig. 8 Improvements over the locally refined mesh: Upper (from left to right) detail of the connectivity matrix for the Standard mesh (●) and the reordered mesh using Sloan (●) and RCM (●) respectively. Lower figure shows the histogram of the distances (defined as in (17)) for the edges of the mesh

is reduced to the 99.8%. Figure 9 shows the connectivity matrices as well as the distribution of the distance for the three buildings in the uniform mesh. It is possible to better appreciate the banded structure of the connectivity matrix for the RCM ordering than in the case of the refined mesh. In the case of the Sloan's technique, another clustering pattern appears. As in the previous mesh, Sloan's ordering increases the number of edges that have got unit distance between their neighbouring cells while RCM amplifies and clusterizes the numbering of the cells improving the amount of pairs with low distance.

Sloan's reordering allows to obtain a large number of pairs with distance one but RCM clusterizes the cells producing a connectivity matrix very close to a banded matrix with two diagonal patterns. The result is that Sloan improves the locality for the pair of neighbouring cells making the operations that are made following the cell ordering more efficient while RCM improves the edge based operations as suggested in (Lacasta et al., 2014b) where it is possible to find that once the cell renumbering is applied, the ordering in the edges pattern improves the operations related to two following edges if they have a very close pair of cells numbering.

Taking the previous considerations into account, the computational effort is analysed. First of all the sequential CPU implementation using the standard numbering for both meshes is stated as the reference computational time. The time required to perform the complete simulation has been of 6.6 minutes in the case of the refined mesh and near to 19.4 hours in the case of the uniform

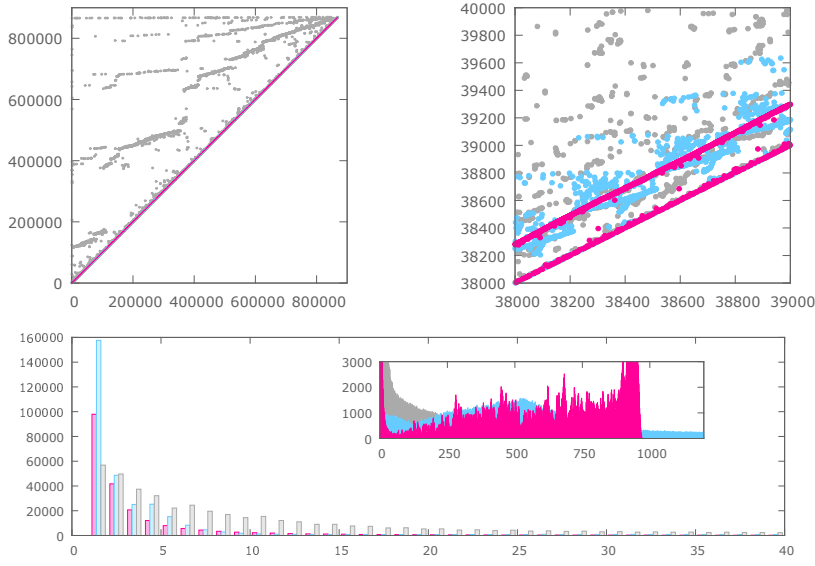


Fig. 9 Improvements over the uniform mesh: Upper (from left to right) detail of the connectivity matrix for the Standard mesh (●) and the reordered mesh using Sloan (●) and RCM (●) respectively. Lower figure shows the histogram of the distances (defined as in (17)) for the edges of the mesh

Case		CPU Sequential	CPU (OpenMP 4 Cores)	
		$t(s)$	$t(s)$	s_{up}
Intel Core i7 4770 @3.40 GHz	<i>Refined</i>	396.72	149.06	2.66
	<i>Uniform</i>	69787.79	30666.26	2.27

Table 2 Detail of computational cost and speed-up for the CPU versions using the most optimized mesh (RCM)

mesh (see Table 2). Not only the amount of elements to be computed but also the smaller time step requirement are the two most important factors that produce such high cost in the case of the uniform mesh. On the other hand, the OpenMP solution relaxes the computational requirement on both refined and uniform meshes, reducing the computational cost of the simulations by 2.66 and 2.27 factor respectively.

Case		GPU Std.		GPU Sloan		GPU RCM	
		$t(s)$	s_{up}	$t(s)$	s_{up}	$t(s)$	s_{up}
Tesla c2070	<i>Refined</i>	39.85	9.95	31.01	12.79	29.44	13.47
	<i>Uniform</i>	3230.42	21.60	2711.05	25.74	2513.64	27.76
Titan Black	<i>Refined</i>	13.19	30.07	12.53	31.66	12.37	32.07
	<i>Uniform</i>	1224.05	57.01	1199.19	58.19	1134.79	61.49

Table 3 Detail of computational cost and speed-up for the compared GPU implementation using the three mesh orderings.

Several considerations must be taken into account when programming the GPU solution (Lacasta et al., 2014a). The performance that this option offers is higher than those for the shared-memory parallel version. The results of the standard ordering, the Sloan based ordering and the RCM ordering in combination with the GPU are detailed in Table 3. Tesla c2075 GPU offers a gain in terms of speed-up that goes from 10 to 21.6 without the optimized way of managing the memory accesses. In this case, the improvement that the ordering methods produce makes possible to increase the performance up to 12.8x in the case of Sloan and 13.5x in the case of RCM for the refined mesh and up to 25x and 27x respectively in the case of the uniform mesh. In other words, the computational effort when applying this optimization reduces in near 25% the required computational time for the same simulations. In the case of the last generation of nVidia GPUs, this reduction is less than in the previous case. It is important to keep in mind the fact that the memory bandwidth in the case of the Titan Black GPU is three times larger than in the case of the Tesla c2075. This means that the improvement with this technique may be the same in percent when comparing the memory access cost, but this GPU not only doubles the computational capacity but also the memory accessing performance. As a result, the overall computational cost is reduced by 10%, producing a variation of only +1x/+4x in the speed-up. It is possible to achieve a speed-up with this GPU in these cases from 30x to 60x depending on the size of the problem. In other words, the simulation that previously costed 19 hours, can be done in less than 19 minutes.

6 Conclusions

In this work, the 2D shallow water equations have been used to simulate a rainfall/runoff event. The numerical method to solve them is a robust finite volume method previously presented as reliable for transient flow over dry irregular topography where wet/dry fronts appear. It has been developed and compared following three programming approaches: the sequential implementation and its adaptation to the Multi-Thread and Many-Core architectures. The Multi-Thread code has been written using the OpenMP library while the adaptation to nVidia GPUs as example of Many-Core technology has been made using CUDA. Moreover, the implementation has been tested over two different GPUs, one released on 2011 and the other one more recent with twice the computing capability. Both are compared with a very recent Intel Core i7 CPU. On the other hand, some improvements on the way the mesh is ordered are applied, showing a reduction on the computational time requirement.

While the CPU parallel version performance has not demonstrated sensitivity to the size of the problem, the performance of the GPU shows a strong variation depending on the size of the problem. The simulation of the same event on a 31000 cells mesh may be accelerated in a 13-32 factor depending on the GPU used. When increasing one order of magnitude the size of the mesh, it is possible to double the speed-up of the GPU solution compared against the

CPU sequential version. The expected tendency is to obtain a larger speed-up as the problem size increases. The limitation is imposed by the data capacity of the GPU card. The combination of MPI and GPU methods could increase both computational efficiency and data capacity.

The improvements on the way the mesh is ordered shows a very high impact on the previous generation nVidia GPUs. Bearing in mind that the last generation of GPUs have multiplied by three the performance on the memory transactions, these ordering methods have improved the memory coalescing, reducing the computational requirement by about 10% using the RCM method on these GPUs. Sloan's algorithm has shown lower performance. While using Sloan's algorithm, the connectivity matrix bandwidth has been reduced, the ordering pattern has been demonstrated not as properly as in the case of the RCM algorithm.

The application of these very useful programming techniques makes possible the simulation of explicit schemes on large spatial resolution cases over long time scales of interest in Hydrology. Moreover, the reduction of the computational time required to perform a simulation makes easier the calibration of parameters that participates in the model. In particular, these improvements allow to calibrate those used in the SCS CN model when it is coupled with the 2D shallow water equations.

References

- Aubry R, Houzeaux G, Vázquez M, Cela JM (2011) "Some useful strategies for unstructured edge-based solvers on shared memory machines", *International Journal for Numerical Methods in Engineering*, 85, 537-561
- Burguete J, García-Navarro P (2001) "Efficient construction of high-resolution TVD conservative schemes for equations with source terms: application to shallow water flows". *International Journal for Numerical Methods in Fluids*, 37, 209-284
- Brodtkorb AR, Stra ML, Altinakar M (2012) "Efficient shallow water simulations on GPUs: Implementation, visualization, verification, and validation" *Computers & Fluids*, 55, 1-12.
- Caviedes-Voullieme D, García-Navarro P, Murillo J (2012) "Influence of mesh structure on 2D full shallow water equations and SCS Curve Number simulation of rainfall/runoff events", *Journal of Hydrology*, 448, 39-59
- Castro MJ, Ortega S, De la Asuncion M, Mantas JM, Gallardo JM (2011). "GPU computing for shallow water flow simulation based on finite volume schemes". *Comptes Rendus Mecanique*, 339(2), 165-184.
- Chandar DD, Sitaraman J, Mavriplis DJ (2013) "A GPU-based incompressible NavierStokes solver on moving overset grids". *International Journal of Computational Fluid Dynamics*, 27(6-7), 268-282.
- Cuthill E, McKee J (1969) Reducing the bandwidth of sparse symmetric matrices In *Proc. 24th Nat. Conf. ACM*, pages 157-172

- Esteves M, Faucher X, Galle S, Vauclin M (2000) "Overland flow and infiltration modelling for small plots during unsteady rain: numerical results versus observed values". *Journal of Hydrology*, 228(3-4),265-282
- Gibbs NE, Poole Jr. WG, Stockmeyer PK (1976) "A Comparison of Several Bandwidth and Profile Reduction Algorithms" *ACM Transactions on Mathematical Software (TOMS)*,2, 322-330
- Giles MB, Burgess DA (1997) "Renumbering unstructured grids to improve the performance of codes on hierarchical memory machines", *Advances in Engineering Software*, 28, 189-201
- Hinkelmann R (2005) "Efficient Numerical Methods and Information-Processing Techniques for Modeling Hydro- and Environmental Systems". *Lecture Notes in Applied and Computational Mechanics*, Vol. 21, Springer-Verlag, Berlin, Heidelberg
- Holwerda T (2007) "Intel: Software Needs to Heed Moore's Law" <http://www.osnews.com/story/17983/Intel-Software-Needs-to-Heed-Moores-Law/>
- Juez C, Murillo J, García-Navarro P (2014) "A 2D weakly-coupled and efficient numerical model for transient shallow flow and movable bed", *Advances in Water Resources*, 71, 93-109,
- Lacasta A, Garcia-Navarro P, Burguete J, Murillo J (2013) "Preprocess static subdomain decomposition in practical cases of 2D unsteady hydraulic simulation", *Computers & Fluids*, 80, 225-232
- Lacasta A, Morales-Hernández M, Murillo J, Garcia-Navarro P (2014a) "An optimized GPU implementation of a 2D free surface simulation model on unstructured meshes", *Advances in Engineering Software*, 78, 1-15
- Lacasta A, Morales-Hernández M, Murillo J, Garcia-Navarro, P (2014b) "Implementation of a GPU 2D shallow water model on unstructured meshes", *International Conference on Computational Methods in Water Resources*, Stuttgart (Germany)
- Lee G, Yu W, Jung K (2013) "Catchment-scale soil erosion and sediment yield simulation using a spatially distributed erosion model", *Environmental Earth Sciences*, 70(1), 33-47.
- López-Barrera D, García-Navarro P, Brufau P, and Burguete, J (2012) "Diffusive-Wave Based Hydrologic-Hydraulic Model with Sediment Transport. II: Validation and Practical Application". *Journal of Hydrologic Engineering* 17:10, 1105-1122
- Lzaro JM, Navarro JS, Gil AG, Romero VE (2014) "Sensitivity analysis of main variables present in flash flood processes. Application in two Spanish catchments: Ars and Aguiln". *Environmental Earth Sciences*, 71(6), 2925-2939
- Morales-Hernández M, Murillo J, García-Navarro P (2013) "The formulation of internal boundary conditions in unsteady 2D shallow water flows: Application to flood regulation". *Water Resources Research*, 49, 471-487
- Mueller C, Martin B, Lumsdaine A (2007) "A comparison of vertex ordering algorithms for large graph visualization" *Visualization*, 2007. APVIS '07. 2007 6th International Asia-Pacific Symposium on , 141-148

- Murillo J, García-Navarro P, Burguete J, Brufau P (2007) "The influence of source terms on stability, accuracy and conservation in two-dimensional shallow flow simulation using triangular finite volumes". *International Journal for Numerical Methods in Fluids*, 54, 543-590
- Murillo J, García-Navarro P, Burguete J (2009a) "Time step restrictions for well-balanced shallow water solutions in non-zero velocity steady states". *International Journal for Numerical Methods in Fluids*, 60,1351-1377
- Murillo J, García-Navarro P, Burguete J (2009b) "Conservative numerical simulation of multi-component transport in two-dimensional unsteady shallow water flow". *Journal of Computational Physics*, 228,5539-5573
- Murillo J, Garcia-Navarro P (2010) "Weak solutions for partial differential equations with source terms: Application to the shallow water equations". *Journal of Computational Physics*, 229, 4327-4368
- Roe PL (1985) "A basis for upwind differencing on the two-dimensional unsteady euler equations". *Numerical Methods for the Euler Equations of Fluid Dynamics*, SIAM
- Sanders BF, Schubert JE, Detwiler RL (2010) "ParBreZo: A parallel, unstructured grid, Godunov-type, shallow-water code for high-resolution flood inundation modeling at the regional scale", *Advances in Water Resources*, 33, 1456-1467
- Serrano-Pacheco A (2009) "Simulacion numrica bidimensional de procesos hidrolgicos e hidraulicos sobre lecho irregular deformable". PhD Thesis.
- Sloan S (1986) "An algorithm for profile and wavefront reduction of sparse matrices". *International Journal for Numerical Methods in Engineering*, 23,13151324
- Smith LS, Liang Q (2013) "Towards a generalised GPU/CPU shallow-flow modelling tool", *Computers & Fluids*, 88:334-343.
- Thibault JC, Senocak I (2009) "CUDA implementation of a Navier-Stokes solver on multi-GPU desktop platforms for incompressible flows". In *Proceedings of the 47th AIAA Aerospace Sciences Meeting* (pp. 2009-758).